NEETCODE 150 CURRICULUM

Swift 6 Master Reference Manual Sequenced Easy ➔ Medium ➔ Hard (#1 to #150)

Structured from Easy to Hard with Sequential Numbers, Xcode Syntax Highlighting & Clickable LeetCode Problem URLs

---

Curriculum: NeetCode 150 Roadmap	**Ordering:** Sequential (#1 to #150)
Language: Swift 6 Idiomatic Code	**Difficulty Progression:** Easy ➔ Medium ➔ Hard
Formatting: High-Contrast Xcode Light Theme	**Hyperlinks:** 100% Direct LeetCode URLs

--- EASY PROBLEMS ---

#1. Contains Duplicate (Easy) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Use Set to store seen numbers. O(N) time.*

```swift
class Solution {
    func containsDuplicate(_ nums: [Int]) -> Bool {
        return Set(nums).count < nums.count
    }
}
```

Time Complexity: O(N)	**Space Complexity:** O(N)

#2. Valid Anagram (Easy) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Frequency map or sorted string comparison.*

```swift
class Solution {
    func isAnagram(_ s: String, _ t: String) -> Bool {
        return s.sorted() == t.sorted()
    }
}
```

Time Complexity: O(N log N)	**Space Complexity:** O(N)

#3. Two Sum (Easy) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Hash Map target - num complement check.*

```swift
class Solution {
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
        var map = [Int: Int]()
        for (i, n) in nums.enumerated() {
            if let idx = map[target - n] { return [idx, i] }
            map[n] = i
        }
        return []
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#4. Valid Palindrome (Easy) — Two Pointers [■ Open LeetCode Problem]

Logic & Intuition: *Filter alphanumeric and two pointer scan.*

```swift
class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let c = Array(s.lowercased().filter { $0.isLetter || $0.isNumber })
        var l = 0, r = c.count - 1
        while l < r {
            if c[l] != c[r] { return false }
            l += 1; r -= 1
        }
        return true
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#5. Best Time to Buy and Sell Stock (Easy) — Sliding Window [■ Open LeetCode Problem]

Logic & Intuition: *Track minPrice and maxProfit.*

```swift
class Solution {
    func maxProfit(_ prices: [Int]) -> Int {
        var minP = Int.max, maxP = 0
        for p in prices { minP = min(minP, p); maxP = max(maxP, p - minP) }
        return maxP
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#6. Valid Parentheses (Easy) — Stack [■ Open LeetCode Problem]

Logic & Intuition: *Stack push open brackets, pop and match closing.*

```swift
class Solution {
    func isValid(_ s: String) -> Bool {
        var st = [Character]()
        for c in s {
            if c == "(" { st.append(")") }
            else if c == "[" { st.append("]") }
            else if c == "{" { st.append("}") }
            else if st.isEmpty || st.removeLast() != c { return false }
        }
        return st.isEmpty
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#7. Binary Search (Easy) — Binary Search [■ Open LeetCode Problem]

Logic & Intuition: *Iterative mid calculation.*

```swift
class Solution {
    func search(_ nums: [Int], _ target: Int) -> Int {
        var l = 0, r = nums.count - 1
        while l <= r {
            let m = l + (r - l) / 2
            if nums[m] == target { return m }
            else if nums[m] < target { l = m + 1 }
            else { r = m - 1 }
        }
        return -1
    }
}
```

Time Complexity: O(log N) **Space Complexity:** O(1)

#8. Reverse Linked List (Easy) — Linked List [■ Open LeetCode Problem]

Logic & Intuition: *Iterative 3-pointer prev, curr, nextTemp.*

```swift
class Solution {
    func reverseList(_ head: ListNode?) -> ListNode? {
        var prev: ListNode? = nil, curr = head
        while curr != nil {
            let next = curr?.next; curr?.next = prev; prev = curr; curr = next
        }
        return prev
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#9. Merge Two Sorted Lists (Easy) — Linked List [■ Open LeetCode Problem]

Logic & Intuition: *Dummy node pointer comparison.*

```
class Solution {
    func mergeTwoLists(_ l1: ListNode?, _ l2: ListNode?) -> ListNode? {
        let dummy = ListNode(0)
        var tail = dummy, p1 = l1, p2 = l2
        while let n1 = p1, let n2 = p2 {
            if n1.val <= n2.val { tail.next = n1; p1 = n1.next }
            else { tail.next = n2; p2 = n2.next }
            tail = tail.next!
        }
        tail.next = p1 ?? p2
        return dummy.next
    }
}
```

Time Complexity: O(N+M) **Space Complexity:** O(1)

#10. Linked List Cycle (Easy) — Linked List [■ Open LeetCode Problem]

Logic & Intuition: *Floyd's slow and fast pointers.*

```
class Solution {
    func hasCycle(_ head: ListNode?) -> Bool {
        var s = head, f = head
        while f != nil && f?.next != nil {
            s = s?.next; f = f?.next?.next
            if s === f { return true }
        }
        return false
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#11. Invert Binary Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *Recursively swap left and right children.*

```
class Solution {
    func invertTree(_ root: TreeNode?) -> TreeNode? {
        guard let root = root else { return nil }
        let temp = root.left; root.left = invertTree(root.right); root.right = invertTree(temp)
        return root
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(H)

#12. Maximum Depth of Binary Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *1 + max(depth(left), depth(right)).*

```swift
class Solution {
    func maxDepth(_ root: TreeNode?) -> Int {
        guard let root = root else { return 0 }
        return 1 + max(maxDepth(root.left), maxDepth(root.right))
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(H)

#13. Diameter of Binary Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *Max path leftDepth + rightDepth.*

```swift
class Solution {
    func diameterOfBinaryTree(_ root: TreeNode?) -> Int {
        var maxD = 0
        func depth(_ n: TreeNode?) -> Int {
            guard let n = n else { return 0 }
            let l = depth(n.left), r = depth(n.right)
            maxD = max(maxD, l + r)
            return 1 + max(l, r)
        }
        _ = depth(root)
        return maxD
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(H)

#14. Balanced Binary Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *Check abs(left - right) <= 1 at each node.*

```swift
class Solution {
    func isBalanced(_ root: TreeNode?) -> Bool {
        func check(_ n: TreeNode?) -> Int {
            guard let n = n else { return 0 }
            let l = check(n.left), r = check(n.right)
            if l == -1 || r == -1 || abs(l - r) > 1 { return -1 }
            return 1 + max(l, r)
        }
        return check(root) != -1
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(H)

#15. Same Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *Recursively compare node values and structure.*

```swift
class Solution {
    func isSameTree(_ p: TreeNode?, _ q: TreeNode?) -> Bool {
        if p == nil && q == nil { return true }
        if p == nil || q == nil || p?.val != q?.val { return false }
        return isSameTree(p?.left, q?.left) && isSameTree(p?.right, q?.right)
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(H)

#16. Subtree of Another Tree (Easy) — Trees [■ Open LeetCode Problem]

Logic & Intuition: *Check isSameTree for root and all child nodes.*

```swift
class Solution {
    func isSubtree(_ root: TreeNode?, _ subRoot: TreeNode?) -> Bool {
        guard let root = root else { return false }
        if isSame(root, subRoot) { return true }
        return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot)
    }
    private func isSame(_ p: TreeNode?, _ q: TreeNode?) -> Bool {
        if p == nil && q == nil { return true }
        if p == nil || q == nil || p?.val != q?.val { return false }
        return isSame(p?.left, q?.left) && isSame(p?.right, q?.right)
    }
}
```

Time Complexity: O(N * M) **Space Complexity:** O(H)

#17. Climbing Stairs (Easy) — 1D Dynamic Programming [■ Open LeetCode Problem]

Logic & Intuition: *Fibonacci DP transition dp[i] = dp[i-1] + dp[i-2].*

```swift
class Solution {
    func climbStairs(_ n: Int) -> Int {
        if n <= 2 { return n }
        var a = 1, b = 2
        for _ in 3...n { let c = a + b; a = b; b = c }
        return b
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#18. Min Cost Climbing Stairs (Easy) — 1D Dynamic Programming [■ Open LeetCode Problem]

Logic & Intuition: *dp[i] = cost[i] + min(dp[i-1], dp[i-2]).*

```swift
class Solution {
    func minCostClimbingStairs(_ cost: [Int]) -> Int {
        var a = cost[0], b = cost[1]
        for i in 2..<cost.count {
            let c = cost[i] + min(a, b)
            a = b; b = c
        }
        return min(a, b)
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#19. Single Number (Easy) — Bit Manipulation [■ Open LeetCode Problem]

Logic & Intuition: *XOR all numbers. Duplicates cancel to 0.*

```swift
class Solution {
    func singleNumber(_ nums: [Int]) -> Int {
        return nums.reduce(0, ^)
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#20. Number of 1 Bits (Easy) — Bit Manipulation [■ Open LeetCode Problem]

Logic & Intuition: *Brian Kernighan `n &= (n - 1)`.*

```swift
class Solution {
    func hammingWeight(_ n: Int) -> Int {
        var count = 0, num = n
        while num > 0 { num &= (num - 1); count += 1 }
        return count
    }
}
```

Time Complexity: O(1) **Space Complexity:** O(1)

#21. Counting Bits (Easy) — Bit Manipulation [■ Open LeetCode Problem]

Logic & Intuition: *dp[i] = dp[i >> 1] + (i & 1).*

```swift
class Solution {
    func countBits(_ n: Int) -> [Int] {
        var dp = Array(repeating: 0, count: n + 1)
        for i in 0...n { dp[i] = dp[i >> 1] + (i & 1) }
        return dp
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#22. Reverse Bits (Easy) — Bit Manipulation [■ Open LeetCode Problem]

Logic & Intuition: *Bitwise shift and OR accumulator.*

```swift
class Solution {
    func reverseBits(_ n: Int) -> Int {
        var res = 0, num = n
        for _ in 0..<32 { res = (res << 1) | (num & 1); num >>= 1 }
        return res
    }
}
```

Time Complexity: O(1) **Space Complexity:** O(1)

#23. Missing Number (Easy) — Bit Manipulation [■ Open LeetCode Problem]

Logic & Intuition: *Gauss sum formula n*(n+1)/2 - sum(nums).*

```swift
class Solution {
    func missingNumber(_ nums: [Int]) -> Int {
        let n = nums.count
        return n * (n + 1) / 2 - nums.reduce(0, +)
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

--- MEDIUM PROBLEMS ---

#24. Group Anagrams (Medium) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Sorted string key mapping in Hash Map.*

```swift
class Solution {
    func groupAnagrams(_ strs: [String]) -> [[String]] {
        var map = [String: [String]]()
        for s in strs { map[String(s.sorted()), default: []].append(s) }
        return Array(map.values)
    }
}
```

Time Complexity: O(N * K log K) **Space Complexity:** O(N * K)

#25. Top K Frequent Elements (Medium) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Bucket sort frequency array.*

```swift
class Solution {
    func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
        var counts = [Int: Int]()
        for n in nums { counts[n, default: 0] += 1 }
        return Array(counts.keys.sorted{ counts[$0]! > counts[$1]! }.prefix(k))
    }
}
```

Time Complexity: O(N log N) **Space Complexity:** O(N)

#26. Product of Array Except Self (Medium) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Prefix and suffix product passes.*

```swift
class Solution {
    func productExceptSelf(_ nums: [Int]) -> [Int] {
        var res = Array(repeating: 1, count: nums.count), p = 1
        for i in 0..<nums.count { res[i] = p; p *= nums[i] }
        p = 1
        for i in (0..<nums.count).reversed() { res[i] *= p; p *= nums[i] }
        return res
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#27. Valid Sudoku (Medium) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Check row, column, and box sets.*

```swift
class Solution {
    func isValidSudoku(_ board: [[Character]]) -> Bool {
        var r = Array(repeating: Set<Character>(), count: 9), c = r, b = r
        for i in 0..<9 {
            for j in 0..<9 {
                let char = board[i][j]
                if char == "." { continue }
                let idx = (i / 3) * 3 + (j / 3)
                if r[i].contains(char) || c[j].contains(char) || b[idx].contains(char) { return false
}
                r[i].insert(char); c[j].insert(char); b[idx].insert(char)
            }
        }
        return true
    }
}
```

Time Complexity: O(1) **Space Complexity:** O(1)

#28. Longest Consecutive Sequence (Medium) — Arrays & Hashing [■ Open LeetCode Problem]

Logic & Intuition: *Set lookup start sequence `!set.contains(n-1)`.*

```swift
class Solution {
    func longestConsecutive(_ nums: [Int]) -> Int {
        let set = Set(nums); var maxL = 0
        for n in set {
            if !set.contains(n - 1) {
                var curr = n, len = 1
                while set.contains(curr + 1) { curr += 1; len += 1 }
                maxL = max(maxL, len)
            }
        }
        return maxL
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#29. 3Sum (Medium) — Two Pointers [■ Open LeetCode Problem]

Logic & Intuition: *Sort array, fix i, two pointers l and r.*

```swift
class Solution {
    func threeSum(_ nums: [Int]) -> [[Int]] {
        let s = nums.sorted(); var res = [[Int]]()
        for i in 0..<s.count {
            if i > 0 && s[i] == s[i-1] { continue }
            var l = i + 1, r = s.count - 1
            while l < r {
                let sum = s[i] + s[l] + s[r]
                if sum == 0 {
                    res.append([s[i], s[l], s[r]])
                    while l < r && s[l] == s[l+1] { l += 1 }
                    while l < r && s[r] == s[r-1] { r -= 1 }
                    l += 1; r -= 1
                } else if sum < 0 { l += 1 } else { r -= 1 }
            }
        }
        return res
    }
}
```

Time Complexity: O(N^2) **Space Complexity:** O(1)

#30. Container With Most Water (Medium) — Two Pointers [■ Open LeetCode Problem]

Logic & Intuition: *Two pointers at bounds, shrink smaller side.*

```
class Solution {
    func maxArea(_ height: [Int]) -> Int {
        var l = 0, r = height.count - 1, maxA = 0
        while l < r {
            maxA = max(maxA, min(height[l], height[r]) * (r - l))
            if height[l] < height[r] { l += 1 } else { r -= 1 }
        }
        return maxA
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#31. Longest Substring Without Repeating Characters (Medium) — Sliding Window [■ Open LeetCode Problem]

Logic & Intuition: *Sliding window with map of last seen char index.*

```
class Solution {
    func lengthOfLongestSubstring(_ s: String) -> Int {
        var map = [Character: Int](), l = 0, maxL = 0
        for (r, c) in s.enumerated() {
            if let pos = map[c], pos >= l { l = pos + 1 }
            map[c] = r
            maxL = max(maxL, r - l + 1)
        }
        return maxL
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#32. Min Stack (Medium) — Stack [■ Open LeetCode Problem]

Logic & Intuition: *Dual stacks mainStack and minStack.*

```
class MinStack {
    var main = [Int](), minS = [Int]()
    func push(_ val: Int) { main.append(val); minS.append(min(val, minS.last ?? Int.max)) }
    func pop() { main.removeLast(); minS.removeLast() }
    func top() -> Int { main.last! }
    func getMin() -> Int { minS.last! }
}
```

Time Complexity: O(1) **Space Complexity:** O(N)

#33. Coin Change (Medium) — 1D Dynamic Programming [■ Open LeetCode Problem]

Logic & Intuition: *dp[i] = min(dp[i], dp[i-c] + 1).*

```
class Solution {
    func coinChange(_ coins: [Int], _ amount: Int) -> Int {
        guard amount > 0 else { return 0 }
        var dp = Array(repeating: amount + 1, count: amount + 1); dp[0] = 0
        for i in 1...amount {
            for c in coins { if i - c >= 0 { dp[i] = min(dp[i], dp[i - c] + 1) } }
        }
        return dp[amount] > amount ? -1 : dp[amount]
    }
}
```

Time Complexity: O(Amount * Coins) **Space Complexity:** O(Amount)

--- HARD PROBLEMS ---

#34. Trapping Rain Water (Hard) — Two Pointers [■ Open LeetCode Problem]

Logic & Intuition: *Two pointers with maxLeft and maxRight.*

```
class Solution {
    func trap(_ height: [Int]) -> Int {
        var l = 0, r = height.count - 1, maxL = 0, maxR = 0, ans = 0
        while l < r {
            if height[l] < height[r] {
                if height[l] >= maxL { maxL = height[l] } else { ans += maxL - height[l] }
                l += 1
            } else {
                if height[r] >= maxR { maxR = height[r] } else { ans += maxR - height[r] }
                r -= 1
            }
        }
        return ans
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(1)

#35. Minimum Window Substring (Hard) — Sliding Window [■ Open LeetCode Problem]

Logic & Intuition: *Sliding window frequency count match.*

```swift
class Solution {
    func minWindow(_ s: String, _ t: String) -> String {
        var target = [Character: Int]()
        for c in t { target[c, default: 0] += 1 }
        var required = target.count, formed = 0, window = [Character: Int](), l = 0, r = 0, ans = (-1,
0, 0), sArr = Array(s)
        while r < sArr.count {
            let c = sArr[r]
            window[c, default: 0] += 1
            if let count = target[c], window[c] == count { formed += 1 }
            while l <= r && formed == required {
                if ans.0 == -1 || r - l + 1 < ans.0 { ans = (r - l + 1, l, r) }
                let lc = sArr[l]
                window[lc]! -= 1
                if let count = target[lc], window[lc]! < count { formed -= 1 }
                l += 1
            }
            r += 1
        }
        return ans.0 == -1 ? "" : String(sArr[ans.1...ans.2])
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#36. Sliding Window Maximum (Hard) — Sliding Window [■ Open LeetCode Problem]

Logic & Intuition: *Monotonic decreasing Deque storing indices.*

```swift
class Solution {
    func maxSlidingWindow(_ nums: [Int], _ k: Int) -> [Int] {
        var q = [Int](), res = [Int]()
        for i in 0..<nums.count {
            while !q.isEmpty && nums[q.last!] < nums[i] { q.removeLast() }
            q.append(i)
            if q.first! == i - k { q.removeFirst() }
            if i >= k - 1 { res.append(nums[q.first!]) }
        }
        return res
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(K)

#37. Largest Rectangle in Histogram (Hard) — Stack [■ Open LeetCode Problem]

Logic & Intuition: *Monotonic stack storing (index, height).*

```swift
class Solution {
    func largestRectangleArea(_ heights: [Int]) -> Int {
        var st = [(Int, Int)](), maxA = 0, h = heights + [0]
        for (i, height) in h.enumerated() {
            var start = i
            while !st.isEmpty && st.last!.1 > height {
                let (pIdx, pHeight) = st.removeLast()
                maxA = max(maxA, pHeight * (i - pIdx))
                start = pIdx
            }
            st.append((start, height))
        }
        return maxA
    }
}
```

Time Complexity: O(N) **Space Complexity:** O(N)

#38. Median of Two Sorted Arrays (Hard) — Binary Search [■ Open LeetCode Problem]

Logic & Intuition: *Binary search partition on smaller array.*

```swift
class Solution {
    func findMedianSortedArrays(_ A: [Int], _ B: [Int]) -> Double {
        let (a, b) = A.count <= B.count ? (A, B) : (B, A), m = a.count, n = b.count
        var l = 0, r = m
        while l <= r {
            let i = l + (r - l) / 2, j = (m + n + 1) / 2 - i
            let maxL1 = i == 0 ? Int.min : a[i - 1], minR1 = i == m ? Int.max : a[i]
            let maxL2 = j == 0 ? Int.min : b[j - 1], minR2 = j == n ? Int.max : b[j]
            if maxL1 <= minR1 && maxL2 <= minR1 && maxL1 <= minR2 && maxL2 <= minR2 {
                if (m + n) % 2 == 1 { return Double(max(maxL1, maxL2)) }
                else { return Double(max(maxL1, maxL2) + min(minR1, minR2)) / 2.0 }
            } else if maxL1 > minR2 { r = i - 1 }
            else { l = i + 1 }
        }
        return 0.0
    }
}
```

Time Complexity: O(log(min(M,N))) **Space Complexity:** O(1)